

TypeScript Modules, Generics and Utility Types for Playwright

Modules

Modules help organize code into reusable files using export and import.

Example:

```
export function waitForPageLoad(page: Page) { ... }  
import { waitForPageLoad } from '../utils/waits';
```

Playwright frameworks commonly use modules for Page Objects, Utilities, Fixtures, and Configurations.

Generics

Generics allow functions, classes, and interfaces to work with multiple data types while maintaining type safety.

Example:

```
function getData<T>(data: T): T { return data; }
```

Usage:

```
const name = getData<string>('Bala');  
const age = getData<number>(30);
```

API Response Example:

```
interface ApiResponse<T> { status:number; data:T; }
```

Utility Types

Partial<T> - Makes all properties optional.

Required<T> - Makes all properties mandatory.

Readonly<T> - Prevents modification of properties.

Pick<T,K> - Selects specific properties.

Omit<T,K> - Removes specific properties.

Record<K,T> - Creates key-value mappings.

ReturnType<T> - Extracts a function's return type.

Example:

```
type UserSummary = Pick<User,'id' | 'name'>;  
type UserWithoutEmail = Omit<User,'email'>;
```

Playwright Framework Usage

Modules: Page Objects, Fixtures, Utilities.

Generics: API Clients, Repositories, Helper Methods.

Utility Types: Test Data Builders, Environment Configurations, Fixture Typing.

Example:

```
class ApiClient {  
  async get<T>(url:string): Promise<T> { ... }  
}
```